

RI.UVT.RO Redesign

Role 6 Implementation Plan

WordPress Developer — API and Content Types

Headless WordPress + React Environment

1. Role Overview

The WordPress Developer — API and Content Types is responsible for building scalable, editable backend structures that power the public React frontend. This role ensures the platform is not composed of static pages only, but of structured content models, reusable data entities, and clean REST API delivery.

2. Phase 1: Content Modeling Preparation

Actual Work

- Analyse content requirements defined by IA and Content roles.
- Identify repeatable entities:
 - Programmes
 - Partnerships
 - Opportunities
 - News items
 - Documents / resources
 - Staff contacts
- Decide which items should become:
 - Standard Pages
 - Posts
 - Custom Post Types
 - Shared taxonomies
 - Custom metadata fields

Deliverable

- Defined backend content architecture and data strategy.

3. Phase 2: Content Types and Field Setup

Actual Work

- Implement structured content types:
 - Custom Post Types
 - Taxonomies
 - Content relationships
- Create structured fields:
 - Titles
 - Descriptions
 - Countries
 - Deadlines
 - Contacts
 - Links
 - Files
 - Status flags
- Ensure content models are:
 - Easy to edit
 - Consistent
 - Expandable over time

Deliverable

- Functional structured content system inside WordPress.

4. Phase 3: REST API Exposure and Dynamic Data Delivery

Actual Work

- Expose required data to the React frontend through REST API endpoints.
- Configure:
 - Standard endpoints
 - Custom Post Type endpoints
 - Field exposure
 - Relationship data

- Ensure dynamic updates:
 - Content changes in WordPress reflect in frontend data output
- Support filtering and structured querying where needed.

Deliverable

- Reliable API-ready backend data layer.

5. Phase 4: Backend Editing Experience

Actual Work

- Optimise editorial workflow:
 - Logical field grouping
 - Clear labels and helper text
 - Minimal confusion in dashboard structure
- Ensure:
 - Non-technical staff can manage content easily
 - Common updates require no developer intervention
 - Editorial governance remains clean

Deliverable

- User-friendly backend editing experience.

6. Phase 5: Reusability and Scalability Enforcement

Actual Work

- Avoid duplicated content logic:
 - Replace repeated structures with shared models
- Ensure:
 - New entries can be added without structural changes
 - Frontend can reuse content patterns consistently
 - Taxonomies remain sustainable long-term
- Refactor inefficient models into reusable systems.

Deliverable

- Scalable backend content system built on reusable logic.

7. Phase 6: Collaboration and Integration

Actual Work

- Work with Role 3:
 - Align content structures with site architecture
- Work with Role 4:
 - Ensure data models support intended layouts
- Work with Role 5:
 - Integrate API data into React routes and components
- Work with Role 7:
 - Support editorial clarity and content governance

Deliverable

- Fully integrated backend system powering the frontend experience.

8. Final Output Responsibilities

- Custom Post Types and taxonomies
- Structured metadata fields
- REST API-ready data architecture
- Scalable reusable backend content models
- Optimised editorial dashboard experience
- Fully integrated WordPress content layer for React frontend

This role ensures the platform evolves from static pages into a scalable content engine where information can grow, adapt, and power the frontend efficiently over time.